

Inbound EDI flow — detailed walkthrough

This walks through exactly what happens to one inbound transmission (a trading partner sending us, say, an X12 873 nomination), end to end, referencing the actual code that does each step. It's the detailed companion to the flow diagram — read this when you need to know *why* a step exists or *which line of code* to change, not just what the shape of the pipeline is.

Every stage below that can fail still returns **HTTP 200** — the outcome is carried inside the signed **request-status** field, never the HTTP status code. The only exceptions are transport-level failures that happen before we know who's even talking to us (body too large, bad auth) — those return a plain, unsigned HTTP error, because there's no protocol-level "who do I address this rejection to" yet.

1. Partner prepares the package (their side)

Before anything reaches us, the trading partner:

1. **Compresses → signs → encrypts** the raw X12 payload in one PGP operation: signed with *their* private key (so we can verify who sent it), encrypted to *our* public key (so only we can read it). This mirrors what `app/crypto/gpg_wrapper.py::GpgService.encrypt_and_sign()` does on our side when we're the sender — same pipeline, opposite direction.
2. **Wraps the resulting ciphertext as multipart/encrypted** (RFC 1847/3156): a **application/pgp-encrypted** control part (literally the text **Version: 1**) followed by the raw, armor-less OpenPGP message as **application/octet-stream**. Our side builds/parses this exact structure in `app/envelope/pgp_mime.py`.
3. **Assembles the outer multipart/form-data envelope**, in the spec-mandated field order (`app/envelope/multipart_codec.py::build_multipart_body()` builds this same structure when we're sending): **from**, **to**, **version**, **receipt-disposition-to**, **receipt-report-type** (the literal string **gisb-acknowledgement-receipt**), **input-format** (X12), **input-data** (the file from step 2, with a **filename=** attribute), then **receipt-security-selection**, and finally the optional/mutually-agreed **transaction-set / refnum / refnum-orig**.
4. **POSTs it** to our **/inbound** endpoint over TLS, with an **Authorization: Basic ...** header (HTTP Basic Auth over TLS is a real NAESB requirement, standards 12.3.14/12.3.28/12.3.29 — not a convenience we bolted on).

2. Gateway receives & authenticates

`app/inbound/routes.py::receive()`, steps in order:

1. **Pre-check Content-Length** against `settings.server.max_body_size_bytes` before touching the body at all — oversized requests get a plain HTTP 413 immediately.
2. **Read the body, then immediately capture time_c = datetime.now(UTC)**. This has to happen *before* auth, parsing, or decryption, because standard 12.3.5 requires the receipt's **time-c** to be generated "by the Receiving Program immediately, prior to further processing" — it's the timestamp that matters for the partner's turnaround-deadline obligations, not whenever we happen to finish validating the request.
3. **Re-check the actual body length** (a **Content-Length** header can lie).

4. **Log the raw request** (method, path, headers with `Authorization` redacted, full body as base64) if `logging.capture_raw_requests` is on — so a transmission that fails before we can make sense of it is still inspectable.
5. **Authenticate** (`app/inbound/auth.py::authenticate_inbound()`): checks the `Authorization` header against every configured partner's `inbound_auth.type`: `basic` is the spec-compliant path; `type: api_key` (Bearer token) is a gateway-only convenience extension with no basis in the standard. No match → plain, unsigned `HTTP 401`. This is deliberately *before* any GPG work, so an unauthenticated caller can't make us spend CPU decrypting garbage.
6. **Assign the sequential `trans_id`** (`tracker.next_trans_id()`, backed by a Postgres sequence, `db/migrations/0002_naesb_receipt_fields.sql`). Per the data dictionary, `trans-id` is "assigned by the Server upon processing before being passed to the decryption process" — so it's handed out once we know a legitimate partner is talking to us, and reused across every receipt this request produces from here on, success or failure.

3. Parse & authorize

1. **`await request.form()`** — Starlette's multipart parser (backed by `python-multipart`) parses the outer envelope. This is deliberate: parsing *untrusted, attacker-controlled* multipart bytes is exactly the kind of thing you want a mature, security-reviewed library doing, not a hand-rolled parser.
2. **`app/envelope/multipart_codec.py::parse_multipart_form()`** pulls out each required field, and for `input-data`, reads the uploaded part and unwraps its inner `multipart/encrypted` structure (`pgp_mime.py::unwrap_pgp_encrypted()`) to recover the raw OpenPGP ciphertext bytes.
3. **Missing or malformed fields raise a typed `EnvelopeError(field, problem)`**, which `app/envelope/error_codes.py::error_code_for_field()` maps to the exact real `EEDM1xx` code — e.g. a missing `to` field is `EEDM101`, an invalid `input-format` is `EEDM107`. This isn't a generic "bad request" bucket; every field/problem combination in the data dictionary has its own documented code.
4. **`from` must equal the authenticated partner's DUNS.** A mismatch (someone authenticated as partner A but claiming to be partner B in the envelope) is `EEDM701` — "Sending party not associated with Receiving party."
5. **If this partner is configured `use_refnum: true`** and didn't include a `refnum`, that's `EEDM119` — "Mutually agreed element, refnum, not present."
6. **Dedup check.** If the partner uses refnum tracking, we key on `(partner, refnum)` — reusing a `refnum` is `EEDM121`, "Duplicate refnum received," matching the spec's own tracking mechanism directly. Otherwise, we dedupe on a SHA-256 digest of the *extracted ciphertext* bytes (not the raw HTTP body — two byte-identical resends can differ in outer multipart boundary/whitespace even though the payload is identical). A digest match here is `GWX-DUPLICATE-DIGEST` — a gateway extension, since content-digest dedup isn't itself a NAESB concept.

4. Record that it's happening

`tracker.create(MessageRecord(...))` inserts a row into Postgres' `messages` table:
`direction=inbound, partner_name, content_digest, transaction_set, input_format,`
`trans_id, refnum, refnum_orig, raw_headers (jsonb), received_at, status=processing.`

The table's `UNIQUE(partner_name, content_digest, direction)` constraint is the real backstop against the dedup check in step 3 — that check and this insert aren't atomic, so a concurrent identical request can race between them. A `UniqueViolation` here is caught and treated exactly like a detected duplicate.

5. Decrypt & verify

1. `gpg.decrypt_and_verify(ciphertext, passphrase)` (`app/crypto/gpg_wrapper.py`) decrypts with our private key and checks the embedded signature in the same GnuPG call.
2. **Decryption failure** → `EEDM699`, "Decryption Error." The spec's own "Pre-validation before Decryption" section acknowledges that GnuPG doesn't always cleanly distinguish "public key invalid" (`EEDM601`) / "not encrypted" (`EEDM602`) / "truncated" (`EEDM603`) from a generic failure, and explicitly sanctions falling back to a generic code when finer classification isn't reliably available — that's what this gateway does.
3. **Signature check:** `signature_valid` must be true *and* the signer's fingerprint must match the fingerprint we imported for this partner at startup. Either condition failing is `EEDM604` — this catches both "not signed at all" and "signed by the wrong key" (e.g. someone else's valid signature) in one check.
4. `app/crypto/policy.py::enforce_policy()` inspects the actual negotiated cipher/digest algorithm IDs (parsed from GnuPG's status-fd `DECRYPTION_INFO/VALIDSIG` lines) against our configured allow-list (`AES256/SHA256` by default). A violation is `GWX-WEAK-ALGO` — this is *our own local security policy*, not a NAESEB mandate (the spec only requires RSA ≥ 2048-bit keys, standard 12.3.26 explicitly disclaims setting site-level algorithm standards beyond that).

6. Deliver the plaintext (sink fan-out)

1. `app/sinks/dispatcher.py::fan_out()` concurrently calls `.deliver()` on every configured sink (filesystem, S3, webhook) with the decrypted plaintext.
2. Filesystem and S3 sinks are `durable: true` by default; webhook is best-effort/non-durable. Both durable sinks use the same path convention, keyed by the *sender's* DUNS:
`{base_dir|prefix}/{partner_duns}/{timestamp}_{digest[:16]}_{transaction_set}.edi`
3. `settings.sinks.require_at_least_one_durable_success` gates acceptance: if zero durable sinks succeeded, the transmission is rejected with `GWX-SINK-FAILURE` even though decryption already succeeded — we won't acknowledge data we didn't actually manage to retain anywhere.
4. Every sink's individual result (ok/error) is recorded on the `messages` row's `sinks_status` jsonb column regardless of the overall outcome.

7. Record & respond

1. `tracker.update_status(..., status="accepted", receipt_verified=True)` finalizes the `messages` row.
2. `NaesbReceipt.ok(server_id, trans_id, time_c=received_at)` (`app/envelope/receipt.py`) builds the four required fields, in the spec-mandated order: `time-c` (`yyyymmddhhmmss`), `request-status` (literally `ok`), `server-id` (from `settings.envelope.server_id`), `trans-id`.

3. `encode_report_part()` renders this as `multipart/report; report-type="gisb-acknowledgement-receipt"`, with `text/html` and `text/plain` sub-parts whose lines are shaped `key=value*` — the spec's literal field-delimiter format, not JSON and not `key: value` text.
4. `gpg.detached_sign()` produces a *detached*, ASCII-armored PGP signature (`application/pgp-signature`) over the exact report bytes. Detached, not embedded/combined, because RFC 1847's `multipart/signed` structure requires the signed content and its signature to sit side by side as independent MIME parts.
5. `build_signed_mime()` wraps both into the final `multipart/signed` envelope (`micalg="pgp-sha256"` by default).
6. Response: `HTTP 200, Content-Type: multipart/signed; ....`

8. Partner confirms delivery

1. The partner extracts the `multipart/report` part's exact bytes (matching what we computed the signature over — no re-serialization in between) and verifies our detached signature against our public key.
2. Reads `request-status: ok` → the transmission is considered settled. An `EEDM###/WEDM###` code → the partner acts on the specific failure (fix the field and resend with a new `refnum`, escalate if it's persistent, etc.).
3. If no response comes back at all (network failure, timeout — not a NAESB-level rejection), standards 12.3.10/12.3.11 require the partner attempt delivery **3 times**; if the gap between the first and last attempt exceeds **30–120 minutes** with no success, that's a spec-defined **Exchange Failure**, and the partner is expected to escalate rather than keep retrying silently. This document describes the flow when *we're* the receiver; our own gateway implements the mirror-image logic when *we're* the sender — see `app/worker.py` and `docs/PLAN.md`'s "Outbound retry / Exchange Failure" row.

What happens after acceptance

- **Audit trail:** the Postgres `messages` row is permanent — status, `trans_id`, timestamps, and per-sink delivery results, whether the transmission was ultimately accepted or rejected.
- **Downstream processing:** the decrypted X12 file sits wherever the sink put it, ready for an internal system (translator, ERP, whatever consumes X12 nominations) to pick up and act on. Reading, parsing, and acting on the X12 *content* itself is explicitly outside this gateway's scope — its job ends at "delivered somewhere durable, with proof."

Error code quick reference

Stage	Failure	Code
Auth	Bad/missing credentials	plain <code>HTTP 401</code> (unsigned, no receipt)
Parse	Missing required field	<code>EEDM1xx</code> (exact code per field, see <code>error_codes.py::FIELD_ERROR_CODES</code>)
Parse	Invalid <code>input-format/version/etc.</code>	<code>EEDM1xx</code> (invalid variant)

Stage	Failure	Code
Authorize	<code>from</code> ≠ authenticated partner	EEDM701
Authorize	Refnum required but missing	EEDM119
Dedup	Refnum reused	EEDM121
Dedup	Ciphertext digest reused (no refnum)	GWX-DUPLICATE-DIGEST
Crypto	Decryption failed	EEDM699
Crypto	Signature missing/invalid/wrong signer	EEDM604
Crypto	Weak cipher/digest/key (local policy)	GWX-WEAK-ALGO
Delivery	No durable sink succeeded	GWX-SINK-FAILURE
Success	—	ok

See also

- [app/inbound/routes.py](#) — the pipeline implementation this document describes.
- [app/envelope/](#) — envelope parsing, MIME wrapping, receipt construction, error codes.
- [docs/PLAN.md](#) — full architecture/design record, including the outbound (sending) side and the retry/Exchange-Failure worker.
- [README.md](#) — spec provenance notes and operational setup.